SpaTE: Learning-Based Traffic Engineering under Sparse Traffic Observation in LEO Satellite Constellations

Linhui Wei, He Ma, Yumei Wang, *Member, IEEE,* and Guangteng Fan

Abstract—Low-Earth-orbit (LEO) satellite constellations require traffic engineering (TE) that reacts quickly to changing topology and demand. Existing learned TE systems achieve low online latency but assume a complete traffic matrix, whereas telemetry and onboard-resource constraints allow a constellation controller to meter only a fraction of demands at each epoch. This paper presents SpaTE, a sparse-aware TE system that allocates all demands directly from incomplete observations. SpaTE combines learnable mask embeddings, asymmetric graph gating, temporal encoding, and cross-attention so that missing measurements are represented explicitly and historical and spatial evidence are fused per demand; a source-neighbor estimate additionally supports optional deployment-time refinement without access to the ground-truth matrix. On a 1,584-satellite Starlink constellation, a five-seed evaluation exposes substantial variation across both observation ratios and training initializations. SpaTE records mixed paired outcomes against zero filling, interpolation, a neighbor-fill baseline, and a TEST-inspired temporal baseline; no overall pairwise comparison is statistically significant. The analysis therefore reports the complete seed-level distributions, identifies the limits of the present sparse-aware design, and separates the contribution of a source-neighbor estimate from the learned allocator. SpaTE runs in under 0.4 s per snapshot and transfers across constellation scales without architectural changes.

Index Terms—LEO satellite constellation, traffic engineering, sparse traffic measurement, maximum link utilization (MLU), graph neural networks.

1 Introduction

Low-Earth-orbit (LEO) satellite constellations are rapidly scaling toward tens of thousands of satellites [1], [2], [3]. Their inter-satellite links (ISLs) carry spatially concentrated and temporally varying traffic under limited capacity [4], [5]. Traffic engineering (TE), which coordinates how each demand is split across candidate paths, is therefore a key control primitive [6], [7], [8], [9]. Classical optimization can take minutes at constellation scale, whereas learned allocators reduce online decision time to the sub-second range by moving most computation offline [10], [11], [12], [9].

Existing learned TE systems nevertheless rely on a complete traffic matrix at every control epoch. This assumption is particularly fragile in an LEO constellation. Measuring every demand requires satellites to meter, aggregate, and transmit fine-grained statistics through telemetry channels that compete with user traffic for ISL and ground-station bandwidth. It also consumes onboard computation, storage, and power, while intermittent ground contacts introduce unavoidable gaps. As illustrated in Fig. 1, only a subset of satellites may meter traffic at a given epoch. The controller then receives a sparse traffic matrix but must still allocate every demand. Efficient TE from sparse observations is thus a natural operating mode for LEO networks, not merely a degraded version of complete measurement.

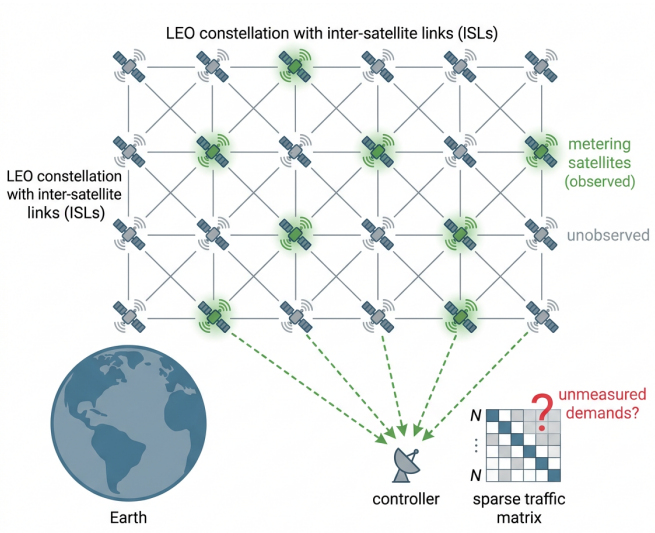

Fig. 1. Sparse traffic observation in an LEO constellation. Only a subset of satellites meters traffic, so the controller receives a sparse traffic matrix while remaining responsible for all demands.

Sparse observation creates three coupled challenges. First, *unmeasured* does not mean *zero*: zero filling suppresses the very demands whose load the controller must anticipate, while explicit matrix completion can propagate reconstruction error into routing. Second, observations have both spatial and temporal structure. Demands originating from

- *L. Wei and G. Fan are with the National Innovation Institute of Defense Technology, Academy of Military Science, Beijing, China (e-mail: weilinhui998@163.com; fanguangteng@163.com).*
- *H. Ma and Y. Wang are with the School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: mahe@bupt.edu.cn; ymwang@bupt.edu.cn).*
- *Manuscript received XXX; revised XXX. (Corresponding author: Guangteng Fan.)*

nearby satellite footprints are correlated, and recent snapshots contain information absent from the current mask. Third, a practical model must remain usable as topology snapshots and active demand sets change, without retraining for every orbital state. TEST [13] shows that temporal learning can help sparse-input TE on terrestrial networks, but sparse observation at constellation scale additionally requires topology-aware representation and size-invariant deployment.

We present SpaTE (Sparse-aware Traffic Engineering), a learned allocator designed around these constraints. It represents each unobserved demand with a learnable mask embedding rather than a fabricated volume. A gated flow-centric GNN lets unobserved demands receive link-state information without injecting their placeholder values into link embeddings. A temporal Transformer encodes recent sparse observations, and per-demand cross-attention fuses temporal and spatial evidence before predicting path split ratios. Shared weights and demand pruning keep the model independent of the instantaneous demand set, enabling reuse across constellation snapshots and scales.

After allocation, SpaTE can optionally refine residual overloads using link loads estimated from the same sparse observation. The important input to this lightweight step is a neighbor estimate: an unmeasured demand is inferred from observed demands sharing its source satellite. This estimate uses geographic locality to avoid granting the controller an unavailable ground-truth traffic matrix. It is an implementation aid to the sparse-aware allocator rather than the organizing principle of the system.

We evaluate SpaTE on a 1,584-satellite Starlink Shell-1 constellation over observation ratios from 2% to 50%. Five paired training seeds reveal that rankings vary with both observation ratio and initialization: SpaTE has mixed outcomes against zero, mean, neighbor, and TEST-inspired alternatives, with no overall pairwise comparison reaching statistical significance. We therefore present the complete seed-level distributions and use the study to distinguish the learned allocator, optional neighbor-assisted refinement, and the regimes in which simple fill-based treatments remain competitive. SpaTE also operates in under 0.4 s per snapshot and transfers to a one-third-scale constellation without changing its architecture.

In summary, this paper makes the following contributions:

- We formulate MLU-oriented TE for dynamic LEO constellations when the controller observes only a fraction of demands but remains responsible for the load induced by all traffic.
- We design a sparse-aware allocator that combines learnable mask embeddings, asymmetric graph gating, temporal encoding, and cross-attention without requiring explicit traffic-matrix completion.
- We exploit source-neighbor traffic correlation to estimate link loads for optional deployment-time refinement using only information available to a sparsely measuring controller.
- We evaluate the design at Starlink scale against zero, mean, neighbor, and TEST-inspired alternatives with seed-level distributions and paired tests, identifying where the present design is competitive and where its advantage is not statistically supported.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formulates the sparse-observation TE problem. Section 4 presents SpaTE. Section 5 reports the evaluation, and Section 6 concludes the paper.

2 Related Work

2.1 ML-Driven Traffic Engineering for WANs

Traffic engineering has long been formulated as a constrained optimization problem [14], [15] solved by linear programming (LP), whose runtime scales poorly with network size. SMORE [16] decouples robust oblivious path selection from online rate adaptation, while NCFlow [17] and POP [18] accelerate LP by problem decomposition, trading allocation quality for speed. A recent line of work replaces online solvers with learned models [19]: DOTE [11] directly maps historical traffic matrices to split ratios with stochastic optimization; Teal [10] combines a flow-centric GNN, multi-agent RL, and ADMM fine-tuning to achieve near-optimal allocations with orders-of-magnitude speedups; FIGRET [20] enhances robustness against traffic uncertainty at per-flow granularity. To survive topology changes without retraining, HARP [12] enforces invariances to node permutation and tunnel reordering, transferring across topologies unseen in training; Aether [21] pursues the same generality with elastic multi-agent graph transformers, and path-based GNNs have been trained for robust distributed TE under link failures [22]. More recently, large language models have been explored as general-purpose traffic planners [23]. However, all of these systems assume the *complete* traffic matrix is available at decision time, an assumption that rarely holds in large-scale satellite networks, where network-wide measurement is prohibitively expensive.

2.2 TE and Routing for Satellite Networks

LEO mega-constellations pose unique challenges for TE: topologies change every few tens of milliseconds as satellites move, and control decisions must be produced within stringent latency budgets [1], [2], [4]. Classical approaches absorb mobility through topology abstractions: the virtual node model binds logical nodes to earth-fixed footprints so that upper-layer routing observes a quasi-static grid [24]. Optimization-based satellite TE has explored segment routing [25], distributed clustering that optimizes elephant flows separately [26], and supervised learning for distributed split-ratio prediction [27]. SaTE [9] is the state-of-the-art learning-based TE for satellite constellations: it formulates a heterogeneous graph over satellites, demands, paths, and links, prunes zero-traffic relations to make training tractable on a single GPU, and selects representative topologies via Graph2Vec and DPP sampling so that one trained model generalizes across topology snapshots with millisecond inference. In parallel, FNC [28], [29] targets dynamic traffic allocation for earth-observation constellations, replacing RL with end-to-end imitation learning and substituting iterative constraint solvers with normalization-based feasibility layers. Notably, FNC's imitation learning requires expert

TABLE 1
Comparison with the related works.

	Teal [10]	SaTE [9]	TEST [13]	FNC [28]	**Ours**
Satellite scenario	×	✓	×	✓	✓
Dynamic topology	×	✓	×	✓	✓
Sparse observation	×	×	✓	×	✓
Temporal modeling	×	×	✓	×	✓
Mask-aware model	×	×	×	×	✓
Neighbor estimate	×	×	×	×	✓
Demand-set drift	×	partial	×	×	✓

demonstrations, and the TE expert is an optimal allocation whose computation needs the complete demand matrix of each episode, a paradigm that fundamentally requires complete post-hoc observability, which is unavailable under sparse measurement. Both SaTE and FNC, like their WAN counterparts, assume the traffic matrix is fully observable at decision time.

2.3 TE with Incomplete Traffic Observations

Network-wide traffic measurement is expensive: collecting a complete traffic matrix requires instrumenting all ingress nodes at fine time granularity, which is particularly onerous on resource-constrained satellite platforms. A classical workaround is a two-stage pipeline: first estimate or complete the traffic matrix via network tomography [30], [31], matrix completion, or compressive sensing [32], then optimize routing on the estimate. However, reconstruction errors propagate to and degrade the downstream allocation. TEST [13] is, to our knowledge, the first end-to-end approach that learns routing policies directly from sparsely measured traffic matrices, integrating a Transformer over historical sparse sequences with a GCN over the topology; it fills unobserved entries with zeros, compensates with synthesized training augmentation, and targets terrestrial networks with static topologies. In the graph learning literature, message passing under missing node features has been addressed via teacher-student distillation [33], but such techniques have not been applied to network TE. No prior work, to our knowledge, performs TE from sparse traffic observations in satellite networks, where the difficulty is compounded by time-varying topologies and drifting demand sets.

2.4 Positioning

Our work sits at the intersection of these three lines (Table 1). Unlike SaTE and FNC, which assume fully observable demands, SpaTE performs TE from sparse observations, down to a 2% observation ratio. Unlike TEST, which fills unobserved entries with zeros and targets static terrestrial networks, SpaTE replaces missing entries with learnable mask embeddings, gates their message passing, and handles LEO dynamics, namely time-varying topologies and drifting demand-pair sets, via demand pruning with size-invariant model weights, enabling train-once, zero-retraining deployment. Finally, SpaTE uses a source-neighbor estimate to support optional congestion-aware refinement without consulting the unobserved ground-truth traffic matrix.

TABLE 2
Main Notations.

Symbol	Definition
$G^t = (V, E^t)$	constellation graph at epoch t; c_e link capacity
D^t, d_i	demand matrix; volume of demand i
$\mathcal{K}^t$	active demand set at epoch t
κ, P_i	candidate paths per demand; path set of demand i
r_i, $f_{i,k}$	split ratios of demand i; flow on path $p_{i,k}$
M, ρ	binary observation mask; observability ratio
O^t, H^t	sparse observation $D^t \odot M$; last-L history
ϕ	learnable mask embedding for unmeasured demands
A, $\tilde{A}$	bipartite adjacency; its mask-gated variant
H_S, H_T, H_F	spatial, temporal, and fused embeddings
π_θ, R	TE policy network; ground-truth reward

3 Model and Problem Formulation

3.1 Network and Traffic Model

We model the constellation at control epoch t as a directed graph $G^t = (V, E^t)$, where V is the set of N satellites and E^t the set of inter-satellite links (ISLs), each link e with capacity c_e. Time is slotted into control epochs; within an epoch the topology snapshot is fixed, while across epochs both E^t and the traffic change as satellites move.

Traffic is described by a demand matrix $D^t \in \mathbb{R}^{N\times N}$, where d_i denotes the volume of the i-th source-destination pair. Since satellite traffic is geographically concentrated, we maintain an *active demand set* $\mathcal{K}^t$, the pairs with nonzero demand in recent history, and only these are modeled. Each demand $i \in \mathcal{K}^t$ is served by κ pre-selected candidate paths $P_i = \{p_{i,1}, \ldots, p_{i,\kappa}\}$ ($\kappa = 4$ edge-disjoint shortest paths in our implementation). The TE decision for demand i is a split-ratio vector $r_i \in \mathbb{R}^\kappa$, allocating flow $f_{i,k} = r_{i,k} \cdot d_i$ to path $p_{i,k}$; the ratios satisfy $r_{i,k} \geq 0$ and $\sum_k r_{i,k} = 1$, so every demand is fully served. Main notations are summarized in Table 2.

3.2 TE with Complete Observation

With the complete demand matrix available, TE is the classic path-based allocation problem. We focus on load balancing, the standard objective in satellite TE, where hot ISLs over populated regions dictate quality of service [9], [25], [26]. It minimizes maximum link utilization (MLU):

$$\min_{\{r_{i,k}\}} \quad \theta \tag{1}$$

$$\text{s.t.} \quad \frac{1}{c_e} \sum_{(i,k):\, e\in p_{i,k}} r_{i,k} d_i \leq \theta, \qquad \forall e \in E^t, \tag{2}$$

$$\sum_{k=1}^{\kappa} r_{i,k} = 1, \qquad \forall i \in \mathcal{K}^t, \tag{3}$$

$$r_{i,k} \geq 0, \qquad \forall i, k, \tag{4}$$

where Eq. (3) enforces flow conservation: every demand must be fully served, so θ cannot be lowered by withholding traffic. The problem is a linear program solvable to optimality, but at constellation scale its solve time exceeds the control epoch [9]. More fundamentally, the link-load term in Eq. (2) requires the volume d_i of every active demand. Sparse-observation TE removes exactly this information from the controller.

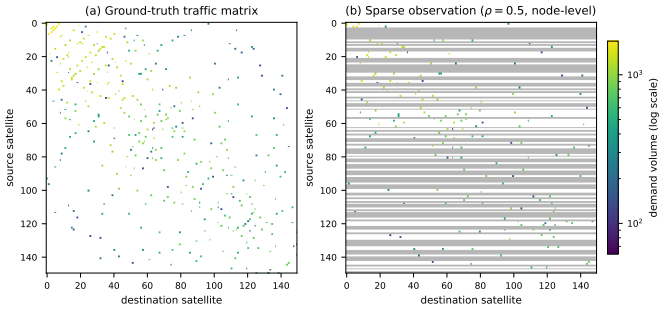

Fig. 2. A ground-truth traffic matrix from the Starlink 22×72 trace (150 busiest satellites shown) and its node-level sparse observation: white cells carry no traffic, gray rows are unmeasured source satellites, and colored cells are observed demands.

3.3 TE with Sparse Observation

3.3.1 Observation Model

At each epoch only a subset of demands is measured. A binary mask $M \in \{0,1\}^{|\mathcal{K}^t|}$ marks demand i as measured ($M_i = 1$) or not ($M_i = 0$), and the controller receives the sparse observation

$$O^t = D^t \odot M, \quad \text{together with } M \text{ itself}, \tag{5}$$

where $\odot$ is the element-wise product. Fig. 2 contrasts a ground-truth traffic matrix from our Starlink trace with its node-level sparse observation. We consider two measurement granularities: *flow-level*, where a fraction ρ of demand pairs is sampled, and *node-level*, where a fraction ρ of satellites meter all their outgoing demands; the latter matches deployments where measurement capability resides on a subset of satellites. The controller additionally retains the last L sparse observations $H^t = (O^{t-L+1}, \ldots, O^t)$.

3.3.2 Problem Statement

A key observation makes decision-making under unknown demands physically executable: the decision variables are split *ratios*, not absolute volumes. Once installed in the forwarding plane, r_i is applied multiplicatively to whatever traffic actually arrives, so a decision for demand i requires no knowledge of d_i. Sparse-observation TE then seeks a policy π that maps the observable state to split ratios for *all* active demands,

$$\{r_i\}_{i\in\mathcal{K}^t} = \pi\left(H^t, M, G^t\right), \tag{6}$$

minimizing the *ground-truth* MLU. Let $f_{i,k}(\pi) = \tilde{r}_{i,k} \cdot d_i$ denote the flow actually delivered on path $p_{i,k}$, where $\{\tilde{r}_i\}$ are the policy's ratios after optional neighbor-assisted rebalancing. The ratios remain conservation-feasible by Eq. (3); the rebalancing only shifts traffic among candidate paths to reduce residual overload. The sparse TE objective is

$$\min_{\pi}\ \mathbb{E}_t\Big[\max_{e\in E^t} \frac{1}{c_e} \sum_{(i,k):\, e\in p_{i,k}} f_{i,k}(\pi)\Big], \tag{7}$$

where the expectation is over traffic and topology dynamics. The link loads are evaluated with the true demands d_i: an unmeasured demand still consumes real capacity even though the policy could not see it, so misjudging it inflates realized MLU. The problem is a partially observable sequential decision problem: unlike Eqs. (1) to (4), where the same D^t appears in both the decision and the constraints, here the decision variables and the evaluation live in different information sets: the policy acts on (O, M) only, while feasibility and performance are settled by the hidden D. Intuitively, the policy is graded against an exam it never gets to read in full.

3.4 Solution Paradigm

Three solution families exist for Eq. (7). *Solve-with-zeros* plugs O directly into the LP in Eqs. (1) to (4), implicitly assuming unmeasured demands are zero; the resulting policy leaves no headroom on the links that unmeasured traffic actually traverses, so realized MLU can be far above the optimum. *Two-stage* approaches first estimate $\hat{D}$ from (H, M), via interpolation, tensor completion, or compressive sensing [30], [31], [32], and then solve the LP on $\hat{D}$; estimation is optimized for reconstruction error, which is misaligned with allocation quality, and errors propagate un-damped into the allocation. We instead adopt *end-to-end learning*: parameterize π_θ as a neural network and optimize θ directly against the TE objective,

$$\max_{\theta}\ \mathbb{E}\big[R\big(\pi_\theta(H, M, G),\, D\big)\big], \tag{8}$$

where R is negative realized MLU from Eq. (7), evaluated on the true D. Since R is non-differentiable through the capacity-constrained network and the true D is available only as a training-time signal, we optimize Eq. (8) with multi-agent reinforcement learning, in which each demand acts as an agent sharing θ and receives a COMA-style marginal-contribution reward. This paradigm makes the observation model in Eq. (5) a property of the *input*, and the optimization target in Eq. (7) a property of the *training signal*, cleanly separating what the policy can see from what it is optimized for.

4 SpaTE Design

4.1 Overview

SpaTE solves the sparse-observation TE problem of Section 3.3 in four stages, as illustrated in Fig. 3. Given the sparse observation (O^t, M) and history H^t, (i) unmeasured demands are represented by a learnable mask embedding (Section 4.2); (ii) a mask-aware gated GNN propagates information between candidate paths and links without letting placeholders contaminate link states (Section 4.3); (iii) a Transformer encodes the observation history and fuses it with spatial embeddings via per-demand cross-attention (Section 4.4); and (iv) a shared policy head outputs split ratios per demand. An optional neighbor-assisted pass refines residual overloads using only an observation-derived estimate (Section 4.6). All learnable weights are shared across demands and independent of the demand-set size, which Section 4.5 exploits for zero-retraining deployment under constellation dynamics. Algorithm 1 summarizes one inference epoch.

Algorithm 1 SpaTE inference at control epoch t

Require: sparse history H^t, mask M, topology G^t, weights $\theta^\star$

Ensure: split ratios $\{r_i\}_{i\in\mathcal{K}^t}$

$\mathcal{K}^t \leftarrow$ active demand set from recent history ▷ pruning, Section 4.5

rebuild bipartite graph and gated adjacency $\tilde{A}$ from $\mathcal{K}^t, M$ ▷ Eq. (10)

$x_i \leftarrow M_i d_i + (1 - M_i)\phi$ for all i ▷ mask embedding, Eq. (9)

$H_S \leftarrow \textsc{GatedFlowGNN}(x, \tilde{A})$ ▷ spatial embeddings

$H_T \leftarrow \textsc{TemporalEncoder}(H^t)$ ▷ temporal embeddings

$H_F \leftarrow \textsc{CrossAttention}(H_T, H_S)$ ▷ Eq. (11)

$\{r_i\} \leftarrow \textsc{PolicyHead}([H_S \,\|\, H_F])$

$\{r_i\} \leftarrow \textsc{NeighborRefine}(\{r_i\}, G^t, \hat{D}^t)$ ▷ optional deployment refinement

return $\{r_i\}$

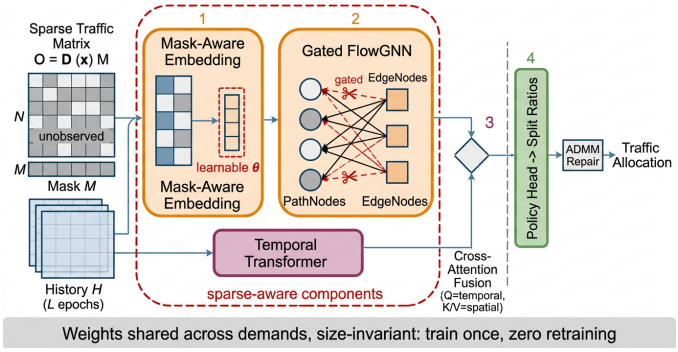

Fig. 3. SpaTE overview: a sparse observation is embedded with mask awareness, processed by a gated flow-centric GNN in parallel with a temporal encoder, fused via per-demand cross-attention, and mapped to split ratios. An optional post-processing pass estimates path congestion from source-neighbor traffic and refines residual overloads without accessing unmeasured demands.

4.2 Mask-Aware Demand Representation

A sparse observation is fundamentally ambiguous: a zero entry may mean "no traffic" or "not measured". Zero-filling, the de-facto treatment in prior sparse TE [13], collapses the two cases and systematically biases the allocator toward starving unmeasured demands. SpaTE instead assigns each unmeasured demand a *learnable mask embedding* $\phi \in \mathbb{R}^\kappa$ (one scalar per candidate path):

$$x_i = M_i \cdot d_i + (1 - M_i) \cdot \phi, \tag{9}$$

so the model learns a data-driven prior for missing traffic during end-to-end training, and, critically, the input itself distinguishes "not measured" from "zero". Learnable imputation has proven effective for incomplete graph features [33]; we bring it to TE, where the imputed value directly steers resource allocation. The embedding is trained jointly with the policy under the TE objective in Eq. (8) rather than a reconstruction loss, so ϕ converges to values that maximize allocation quality, not reconstruction accuracy.

4.3 Gated Message Passing

SpaTE inherits Teal's flow-centric graph [10]: candidate paths and physical links form a bipartite structure where a PathNode connects to the EdgeNodes it traverses, and message passing alternates between them with GCN-style normalized aggregation [34]. Under sparse observation, however, symmetric message passing is harmful: placeholder values on unmeasured PathNodes would propagate into EdgeNode embeddings and corrupt the network-wide estimate of link utilization, a phenomenon consistent with the feature-structure interference observed in incomplete graphs [33]. We therefore gate the propagation *asymmetrically*. Let A be the normalized bipartite adjacency; we zero out entries that carry path-to-edge messages from unmeasured paths while keeping all edge-to-path entries:

$$\tilde{A}[e,p] = M_p \cdot A[e,p], \qquad \tilde{A}[p,e] = A[p,e], \tag{10}$$

where M_p is the demand-level mask lifted to path p. Unmeasured demands thus remain *listeners*: they sense link states, which are shaped by measured traffic, but inject no fabricated volume into them. The gate is a fixed binary function of the mask with no extra parameters, adding zero inference overhead. The asymmetric gate yields a formal isolation property:

Proposition 1 (Placeholder isolation). Under the gated propagation in Eq. (10), the embeddings of all EdgeNodes and of all PathNodes belonging to measured demands are independent of the mask embedding ϕ; ϕ influences only the allocations of unmeasured demands.

Proof: Consider first the spatial stream, by induction on GNN layers. At layer 0, only unmeasured PathNodes carry ϕ. At layer l, an EdgeNode aggregates path-to-edge messages with weights $\tilde{A}[e,p] = M_p A[e,p]$, which vanish for every unmeasured path; hence EdgeNode embeddings depend only on measured-path inputs and their own history, none of which contains ϕ. A measured PathNode aggregates EdgeNode embeddings (ϕ-free by the above) and its own ϕ-free input; the per-demand layers mix only paths of the same demand, which share the same mask value. The temporal and fusion streams preserve the property: the temporal encoder acts per path on the raw observation history H^t, which contains no ϕ by construction, and the cross-attention of Eq. (11) operates strictly within each demand, so the fused features of a measured demand combine only its own ϕ-free temporal and spatial embeddings. Thus ϕ never enters any ϕ-free branch, and only unmeasured PathNodes, and consequently their own split ratios, depend on ϕ. □

In other words, learning a poor prior ϕ can at worst misallocate the unmeasured demands themselves, but can never corrupt the allocations of measured traffic, a robustness guarantee that zero-filling with ungated propagation does not provide.

4.4 Temporal-Spatial Fusion

Satellite traffic exhibits strong short-term temporal correlation; the recent history of sparse observations therefore carries recoverable information about currently missing entries. SpaTE encodes, for each candidate path, its observation

sequence over the last L epochs with a lightweight Transformer [35]: scalar volumes are log-compressed, which we find essential for training stability, and projected to d_{model} dimensions, summed with learnable positional embeddings, and passed through a Transformer encoder; the representation at the latest step forms the temporal embedding H_T. Rather than concatenating temporal and spatial features naively, we fuse them with *per-demand cross-attention*: within each demand, temporal embeddings of its κ candidate paths act as queries attending to the κ spatial (GNN) embeddings H_S as keys and values,

$$H_F = \text{softmax}\left(\frac{W_q H_T \,(W_k H_S)^\top}{\sqrt{d}}\right) W_v H_S, \qquad (11)$$

and the fused features $[H_S \,\|\, H_F]$ feed the policy head. Intuitively, the temporal stream decides *which* spatial evidence to trust for each path: when the current entry is missing, attention shifts toward paths and links whose states corroborate the historical pattern. The full pipeline infers in under 0.4 s per snapshot on the 1,584-satellite constellation, well within the 5-minute control-loop period.

4.5 Scaling to Dynamic Constellations

4.5.1 Demand Pruning

Modeling all $N(N-1)$ pairs of a 1,584-satellite constellation yields 2.5M demands and tens of millions of path variables, which is infeasible for online control. Satellite traffic, however, is geographically concentrated: only 0.25% of pairs ever carry traffic in our dataset. SpaTE instantiates PathNodes only for the demand-pair union observed in historical traffic matrices, shrinking the graph by $\sim 400\times$ with no loss of served traffic, in the same spirit as SaTE's traffic pruning [9].

4.5.2 Zero-Retraining under Drift

Pruned demand sets are not static: as satellites move, the active pair set drifts (we measure $\sim 10\%$ turnover across our trace). Rather than retraining, SpaTE exploits a structural property of its architecture: every learnable component (GNN layers, mask embedding, temporal encoder, fusion and policy heads) is shared across demands and independent of the demand-set size. The demand set, candidate paths, and the bipartite graph are runtime *inputs*, not parameters. At each control epoch the controller rebuilds the graph from the current active set (a millisecond-scale index operation on +Grid constellations, where candidate paths depend only on the orbital displacement between endpoints and reduce to $O(N)$ path templates) and reuses the trained weights as-is. Demands outside the modeled set, necessarily sporadic mice flows, fall back to shortest-path routing until the next set refresh. This reuse is justified by the following structural property:

Proposition 2 (Size invariance and permutation equivariance). The parameter set of SpaTE is independent of $|\mathcal{K}^t|$, N, and $|E^t|$, and the mapping from inputs to split ratios is equivariant to any permutation of the demand indices.

Proof: Every learnable component is a per-node or per-demand operator with shared weights: the mask embedding ϕ and temporal encoder act on each path token, the GNN layers combine node-wise linear maps with sparse aggregation, the fusion and policy heads act within each demand. None of their parameter shapes involves $|\mathcal{K}^t|$, N, or $|E^t|$. Permuting demand indices permutes the rows of every input and of the bipartite adjacency consistently; since shared node-wise operators commute with permutations and sparse aggregation is index-based, all intermediate embeddings, and hence the outputs, are permuted identically. □

4.6 Training and Neighbor-Assisted Refinement

SpaTE trains with the multi-agent RL scheme of Teal [10]: each demand is an agent sharing the policy network. The policy outputs a Gaussian mean per path, and actions are sampled during training. To assign credit, each agent receives a COMA-style [36] counterfactual reward that estimates its marginal contribution to the global objective in Eq. (8): for demand i,

$$A_i = \mathbb{E}_{\tilde{r}_i \sim \pi'}\big[R(r_{-i}, r_i) - R(r_{-i}, \tilde{r}_i)\big], \qquad (12)$$

where r_{-i} denotes the allocations of all other demands and $\tilde{r}_i$ is a counterfactual action resampled from a reference distribution π'; the policy gradient weights each agent's log probability by A_i. Algorithm 2 summarizes offline training. Two points deserve emphasis. First, the reward is computed against *ground-truth* demands during offline training (sparse observation constrains the policy input, never the training signal), which is exactly what a simulator or historical trace provides. Second, we deliberately retain reward-driven RL rather than imitation learning [28]: imitation learning requires expert allocations solved from *fully realized* demands of each episode, a supervision signal that is unavailable when even post-hoc observation is sparse; RL needs only the reward, which the network itself provides.

Neighbor-assisted refinement. The policy head uses a softmax, so its split ratios satisfy flow conservation in Eq. (3) by construction. A short post-processing pass may nevertheless improve MLU by moving a fraction of each demand from its most congested candidate path to its least congested one. Each pass preserves the per-demand total and hence cannot reduce MLU by dropping traffic. Our deployed setting uses two rounds, each containing ten lightweight sweeps.

The key difficulty under sparse observation is not the rebalancing rule but how to estimate the link loads used to rank paths. A controller that meters only a fraction ρ of demands cannot compute true utilization. Consequently, a refinement implementation that reads the ground-truth matrix (denoted ORACLE) is unsuitable for deployment. SpaTE computes $\hat{D}^t$ from the observation instead: every unobserved demand is assigned a *neighbor estimate* from observed demands sharing its source satellite, scaled by the observed per-source traffic. This choice exploits geographic locality in satellite traffic and is used only to estimate path congestion; the learned allocator remains the main decision module. The evaluation compares this estimate with zero filling and the undeployable oracle to isolate the value of the traffic estimate rather than presenting post-processing as the central contribution.

Algorithm 2 Offline training of SpaTE

Require: historical trace $\{(D^t, G^t)\}$, mask M, learning rate η
Ensure: trained weights $\theta^\star$ (incl. mask embedding ϕ)
build demand set $\mathcal{K}$ and graph from training-slice TMs
for each epoch **do**
  for each snapshot t in the training slice **do**
    form sparse input (H^t, M); sample actions $r \sim \pi_\theta(\cdot \mid H^t, M, G^t)$
    compute counterfactual rewards $\{A_i\}$ via Eq. (12) **using the true** D^t
    $\theta \leftarrow \theta + \eta \sum_i A_i \nabla_\theta \log \pi_\theta(r_i)$
  end for
end for
return $\theta^\star$

4.7 Complexity Analysis

Let $D = |\mathcal{K}^t|$ be the number of active demands, κ the candidate paths per demand, E the number of ISLs, and $\bar{\ell}$ the average path length. The gated FlowGNN performs message passing over the bipartite graph whose edge count is $O(D\kappa\bar{\ell})$; with Λ layers its cost is $O(\Lambda D\kappa\bar{\ell}d)$ for hidden width d. The temporal encoder applies a Transformer of length L independently to each of the $D\kappa$ path tokens, costing $O(D\kappa L^2 d)$, and the per-demand cross-attention over κ paths adds $O(D\kappa^2 d)$. Since κ, L, $\bar{\ell}$, and d are small constants fixed by design, the end-to-end inference cost is *linear* in the number of active demands, $O(D)$, and all operations are batched across demands on the GPU. Crucially, the parameter count is independent of D, N, and E: the same weights apply to any demand set, which is what enables zero-retraining reuse (Section 4.5). In our configuration the entire model has merely 4,468 trainable parameters, small enough to be replicated on resource-constrained satellite platforms. Demand pruning reduces D from $N(N-1)$ to the observed support (roughly $400\times$ on our constellation), turning an otherwise intractable problem into one solved well within the control period.

5 Evaluation

5.1 Setup

Constellation. Our main testbed is a Starlink Shell-1 Walker constellation [3] interconnected in a +Grid pattern (Table 3); orbital dynamics are derived from the Satellite Tool Kit (STK). With 1,584 satellites, an average path length of 23.51 hops, and a diameter of 47, capacity conflicts between demands are far more frequent than in the terrestrial WANs used in prior TE studies [10]. For the cross-scale check (Section 5.5) we use a one-third-scale instance of the same architecture (24 planes × 22 satellites, 528 nodes).

Traffic. Traffic matrices are generated by random city sampling [37]: cities are drawn with probability proportional to their population, each sampled city pair is attached to its nearest satellites, and per-pair flows are aggregated into satellite-level demands (Table 4). We use 101 consecutive snapshots (one every 5 minutes), with snapshots 0–79 for training, 80–89 for validation, and 90–100 for testing, a strict temporal split with no leakage. ISL capacity is calibrated so that the fully-observed optimum leaves moderate congestion, keeping TE decisions non-trivial. Demand pruning (Section 4.5) restricts the modeled demand set to the ∼6.3K pairs that ever carry traffic, out of 2.5M node pairs.

TABLE 3
Simulation parameters of the Starlink constellation.

Parameter	Value
Height of LEO satellites	550 km
Constellation configuration	Walker
Total number of satellites	1,584
Number of orbital planes	72
Satellites per plane	22
Inclination of orbits	55°
Phase factor	3
Inter-satellite links	6,336

TABLE 4
Statistics of the population-based traffic trace (Mbps).

Metric	Value
Snapshots (5-min interval)	101
Nonzero demand, max	1,800
Nonzero demand, mean±std	159.5 ± 242.3
Total demand per snapshot	890,242
Active pairs per snapshot	5,583
Active-pair union (trace)	6,334

Metric and protocol. Our metric is the *realized MLU* evaluated against ground-truth demands (lower is better); flow conservation in Eq. (3) is enforced by construction, so MLU cannot be gamed by under-serving traffic. We sweep the observation ratio $\rho \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$ and run five random training seeds per configuration. Unless stated otherwise, every method is paired with the *identical deployable post-processing layer*: two rebalancing sweeps whose utilization estimate reads only the sparse observation with neighbor fill (OBS-NBR, Section 4.6), so that no method receives unavailable demand information and the allocator remains the only factor that varies. The ORACLE variant, which reads the ground-truth matrix, appears only in the neighbor-estimation diagnostic. Each point reported below is the mean MLU over the same eleven chronological test snapshots for one training seed. We show all five seed points and summarize them by mean±standard deviation, median, and range. Because the snapshots in a trace are temporally correlated, headline comparisons are paired on (ρ, seed), not on snapshots, and are assessed with two-sided sign tests.

Baselines. All learned baselines share the training pipeline, path sets, and optional neighbor-assisted refinement of SpaTE and differ only in how they handle sparsity:

- *zero-fill*: the sparse adaptation of full-observation systems such as Teal [10] and SaTE [9], which cannot run on an incomplete matrix; unobserved entries are set to zero and the model is trained end to end on the filled input. SaTE's heterogeneous-graph allocator targets the same MLU objective with the same pruning, so this variant is its closest sparse-input proxy.
- *mean-interp*: a two-stage complete-then-optimize

TABLE 5
Model and training hyperparameters (defaults held fixed across all ρ and seeds).

Parameter	Value
FlowGNN layers Λ	6
Candidate paths κ	4
Temporal d_{model} / heads / layers	16 / 2 / 1
Fusion dim d_{fuse}	8
History length L	3
COMA samples per update	30
Mask-embedding init	57.0 (median nonzero demand)
Learning rate	1×10^{-4}
Epochs (warmup / early stop)	60 (4 / on)
Restart screening	3 inits
Batch size	20
Untrained control	0 epochs, 1 init
Neighbor refinement (deployed)	2 sweeps-of-ten
Trainable parameters	4,468

baseline that fills unobserved entries with the mean of observed demands before allocation.

- *nbr-fill*: a stronger two-stage baseline that fills each unobserved demand from observed demands sharing its source satellite, the same estimate used by the optional refinement.
- *TEST-inspired*: a controlled baseline motivated by TEST [13], with zero-filled input, a Transformer over the observation history ($L = 3$), and an ungated GCN. We use the same paths, objective, and training pipeline as the other methods rather than claiming reproduction of TEST's original terrestrial setup.
- *untrained*: the SpaTE architecture with weights frozen at random initialization, isolating the contribution of training. The LP optimum is used only as an information-theoretic reference, not as an online baseline.

Implementation. Table 5 lists the hyperparameters, drawn from the code defaults and held fixed across all ρ and seeds. The untrained control shares the trained model's random initialization and differs only in skipping gradient updates (`epochs=0`); it is therefore a clean ablation of the *training* factor with identical architecture, mask embedding, and post-processing.

5.2 Seed-Level Behavior of SpaTE

We first examine the complete deployed pipeline itself, rather than starting with a ranking against other methods. Fig. 4 plots all five seed-level test averages for SpaTE and its untrained counterpart under the common neighbor-assisted refinement. Training wins 14 of the 25 matched (ρ, seed) pairs, loses 11, and has no ties (two-sided sign test $p = 0.690$). At $\rho = 0.5$, for example, the mean is 1.639 ± 0.074 after training versus 1.757 ± 0.107 without training; at the other ratios the paired outcomes are mixed. Thus these data do not establish a statistically significant end-to-end training advantage, but they expose the seed sensitivity that a single aggregate score would hide.

We next test the three representation choices without post-processing at the two sparsest ratios, where their effect is least likely to be hidden by rebalancing. Fig. 5 reports every available seed. The outcomes are mixed: no configuration dominates the others at both ratios, and the three-seed diagnostic is insufficient for a component-level performance claim. We therefore retain the placeholder-isolation result of proposition 1 as a structural property, rather than interpreting it as proof that gating, masking, or temporal encoding always lowers MLU.

5.3 Sensitivity to the Neighbor Estimate

The optional refinement pass needs an estimated traffic matrix only to rank candidate paths by congestion. Fig. 6 fixes the untrained control at $\rho = 0.3$ and shows all three diagnostic seeds. Moving from no post-processing (median 2.833) to an observation with zeros elsewhere (2.295), and then to the source-neighbor estimate (1.740), substantially changes the congestion signal available to the rebalancer. The oracle reference reaches 1.599, but uses unavailable ground-truth traffic and five sweeps; it is a ceiling reference rather than a protocol-matched baseline. The deployable conclusion is therefore limited to the value of the neighbor estimate over zero filling, not a claim that post-processing is the central learned mechanism.

5.4 External Comparison

After the internal checks, we compare with external sparsity treatments. Fig. 7 exposes all 150 seed-level results, while Table 6 gives their numerical summary. No method has a uniform advantage across observation ratios. In particular, SpaTE records 16 wins and 9 losses against zero filling, but the paired sign test gives $p = 0.2295$; its comparisons with TEST-inspired, mean-interp, nbr-fill, and untrained controls are likewise non-significant (Table 7). Hence we report the observed rankings and variability rather than claim overall superiority.

5.5 Cross-Scale Generality

To test whether the allocator depends on constellation scale, we reuse its size-invariant architecture on a one-third-scale Shell-1 instance with 528 satellites and 2,112 ISLs. Table 8 isolates the learned allocation prior without post-processing at two observation ratios. At $\rho = 0.05$, training lowers median MLU from 4.02 to 3.04 and wins all three paired seeds. The result at $\rho = 0.3$ is mixed, indicating that the most informative observation regime shifts with traffic density. The model requires no architectural change as the graph and demand-set sizes change, which supports reuse across constellation scales while avoiding a claim of uniform gains.

5.6 Inference Latency

On the 1,584-satellite constellation, one complete allocation pass (demand-set assembly, graph rebuild, gated GNN, temporal encoder, fusion, policy head, and optional neighbor-assisted refinement) completes in under 0.4 s per snapshot on a single GPU, against a 5-minute control period. Combined with the $O(D)$ complexity of Section 4.7 and the 4,468-parameter model size, SpaTE fits the compute envelope of a constellation controller.

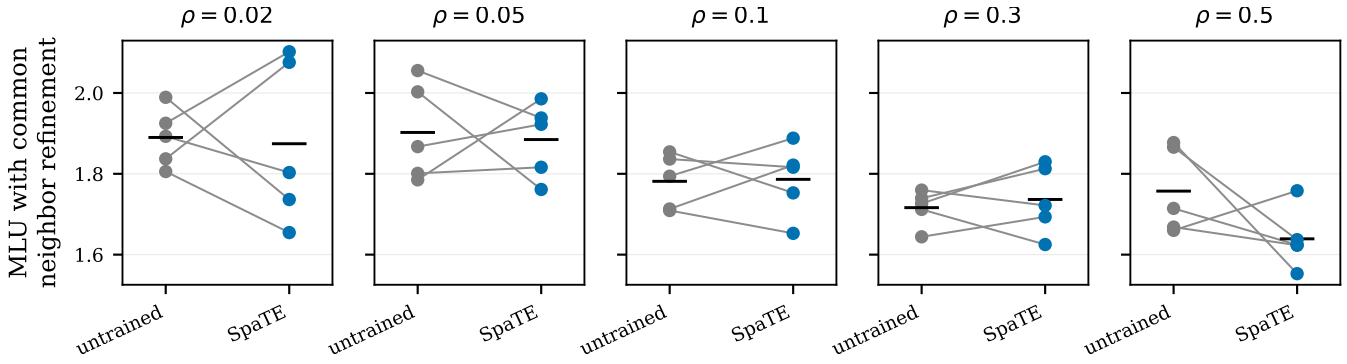

Fig. 4. Internal check of the deployed pipeline. Every marker is one training seed averaged over the same 11 chronological test snapshots; a gray segment pairs identical seeds and a black bar marks the mean. The common neighbor-assisted refinement is enabled for both endpoints.

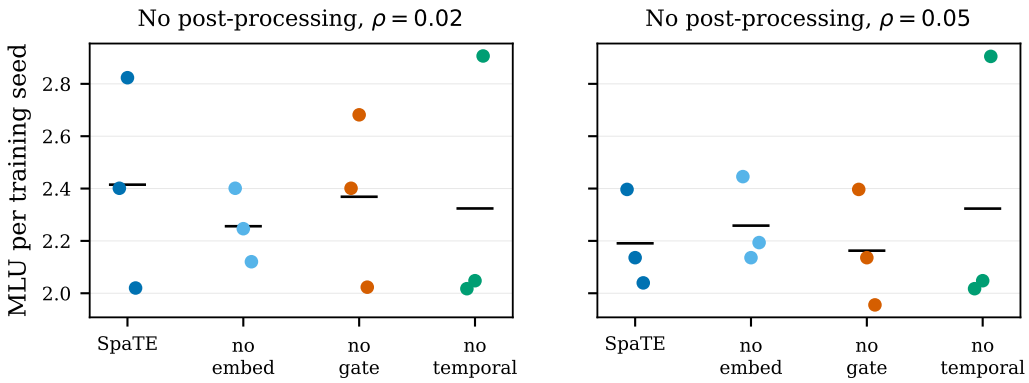

Fig. 5. Low-observability component diagnostic without post-processing (three seeds). Dots are seed-level test averages and black bars are means. The mixed ordering motivates reporting the module choices as design alternatives rather than claiming a uniform component gain.

TABLE 6
External comparison under the aligned deployable protocol. Entries are mean±standard deviation across five training seeds; Fig. 7 shows every seed value and the exported supplementary CSV gives medians and ranges. Lower MLU is better.

	observation ratio ρ				
Method	0.02	0.05	0.1	0.3	0.5
SpaTE	1.874 ± 0.203	1.885 ± 0.093	1.786 ± 0.089	1.737 ± 0.085	1.639 ± 0.074
TEST-inspired [13]	1.809 ± 0.100	1.833 ± 0.145	1.773 ± 0.083	1.769 ± 0.056	1.639 ± 0.127
zero-fill [10], [9]	1.805 ± 0.026	1.876 ± 0.061	1.856 ± 0.065	1.719 ± 0.038	1.705 ± 0.142
mean-interp	1.877 ± 0.152	1.840 ± 0.035	1.939 ± 0.103	1.732 ± 0.050	1.682 ± 0.091
nbr-fill	1.756 ± 0.079	1.867 ± 0.038	1.839 ± 0.078	1.745 ± 0.054	1.680 ± 0.114
untrained	1.890 ± 0.073	1.902 ± 0.121	1.781 ± 0.068	1.716 ± 0.044	1.757 ± 0.107

TABLE 7
Paired comparison of SpaTE against each external control over 25 (ρ, seed) pairs. A win means lower MLU.

Control	Wins–losses	Ties	p
TEST-inspired	11–14	0	0.6900
zero-fill	16–9	0	0.2295
mean-interp	16–9	0	0.2295
nbr-fill	15–10	0	0.4244
untrained	14–11	0	0.6900

TABLE 8
Cross-scale check on the 528-satellite instance without post-processing (median MLU and paired training gain over 3 seeds).

ρ	Trained	Untrained	Paired gain	Wins
0.3	3.69	3.83	−3.2%	1/3
0.05	3.04	4.02	+24.4%	3/3

5.7 Takeaways

- The complete five-seed distribution, not a column-wise median, is required to interpret sparse-observation TE: none of SpaTE's external pairwise

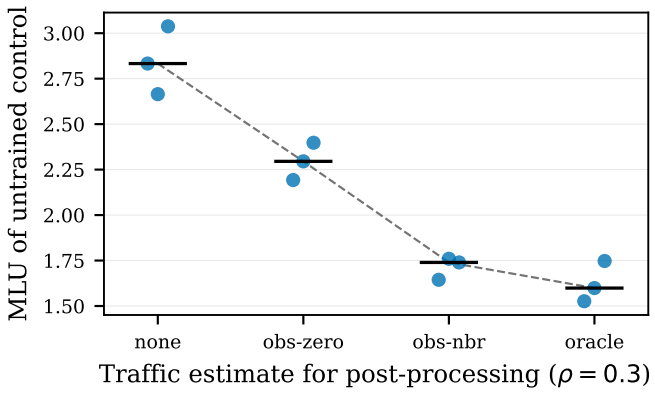

Fig. 6. Seed-level diagnostic for the traffic estimate used by post-processing at $\rho = 0.3$. Black bars denote medians. The oracle point is an unavailable five-sweep reference; deployed methods use the observation-derived estimate.

comparisons is statistically significant.

- Training and the three representation choices show mixed outcomes in the available internal diagnostics; the architecture's structural properties should not be read as a universal performance guarantee.
- Source-neighbor estimation provides a substantially more useful deployable congestion signal than zero filling in the $\rho = 0.3$ diagnostic, while the oracle remains an unavailable reference.

6 Conclusion

This paper studied MLU-oriented traffic engineering for LEO satellite constellations when telemetry constraints expose only a small fraction of demands. We presented SpaTE, a sparse-aware learned allocator that represents missing measurements explicitly, prevents placeholder traffic from contaminating graph messages, and combines spatial and temporal evidence before predicting path split ratios. Shared parameters and demand pruning keep the architecture independent of the instantaneous demand-set size.

On a 1,584-satellite Starlink constellation, five paired training seeds show that MLU rankings vary with both the observation ratio and initialization. SpaTE records more paired wins than zero-fill, mean-interp, and nbr-fill controls, but none of its external pairwise comparisons is statistically significant. The component diagnostic is also mixed at the two sparsest ratios. These negative results matter: they show that the structural plausibility of a sparse-aware representation does not by itself establish a uniform end-to-end MLU gain, and motivate reporting all seed-level outcomes rather than only a column-wise summary statistic.

For deployment, a source-neighbor estimate can provide optional post-processing with a useful congestion signal without access to the complete traffic matrix. It is a supporting mechanism, not a substitute for the sparse-aware allocator. Future work should improve robustness across initializations and observation masks through adaptive measurement scheduling, observation-aware model selection, and objectives that explicitly control worst-case MLU.

References

[1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*, 2018.

[2] D. Bhattacherjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.

[3] G. Giuliari, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, "Internet backbones in space," *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, 2020.

[4] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the "internet from space" with hypatia," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.

[5] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at starlink performance," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.

[6] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013, pp. 3–14.

[7] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.

[8] C.-Y. Hong, S. Mandal, M. Al-Fares, M. Zhu, R. Alimi, K. N. Bollineni, C. Bhagat, S. Jain, J. Kaimal, S. Liang, K. Mendelev, S. Padgett, F. Rabe, S. Ray, M. Tewari, M. Tierney, M. Zahn, J. Zolla, J. Ong, and A. Vahdat, "B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined WAN," in *Proceedings of the ACM SIGCOMM 2018 Conference*, 2018, pp. 74–87.

[9] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, "SaTE: Low-latency traffic engineering for satellite networks," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 896–916.

[10] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of WAN traffic engineering," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.

[11] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, "DOTE: Rethinking (predictive) WAN traffic engineering," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1557–1581.

[12] A. A. AlQiam, Y. Yao, Z. Wang, S. S. Ahuja, Y. Zhang, S. G. Rao, B. Ribeiro, and M. Tawarmalani, "Transferable neural WAN TE for changing topologies," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 86–102.

[13] Y. Guo, Z. Huang, M. Ding, and H. Luo, "An end-to-end learning approach for traffic engineering with sparse traffic measurements," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 5448–5463, 2026.

[14] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM 2000*, 2000.

[15] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, "COPE: Traffic engineering in dynamic networks," in *Proceedings of the ACM SIGCOMM 2006 Conference*, 2006.

[16] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C. L. Lim, and R. Soulé, "Semi-oblivious traffic engineering: The road not taken," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 157–170.

[17] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, and P. Bailis, "Contracting wide-area network topologies to solve flow problems quickly," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 175–200.

[18] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, "Solving large-scale granular resource allocation problems efficiently with POP," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 521–537.

[19] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, 2017.

[20] X. Liu, S. Zhao, Y. Cui, and X. Wang, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 117–135.

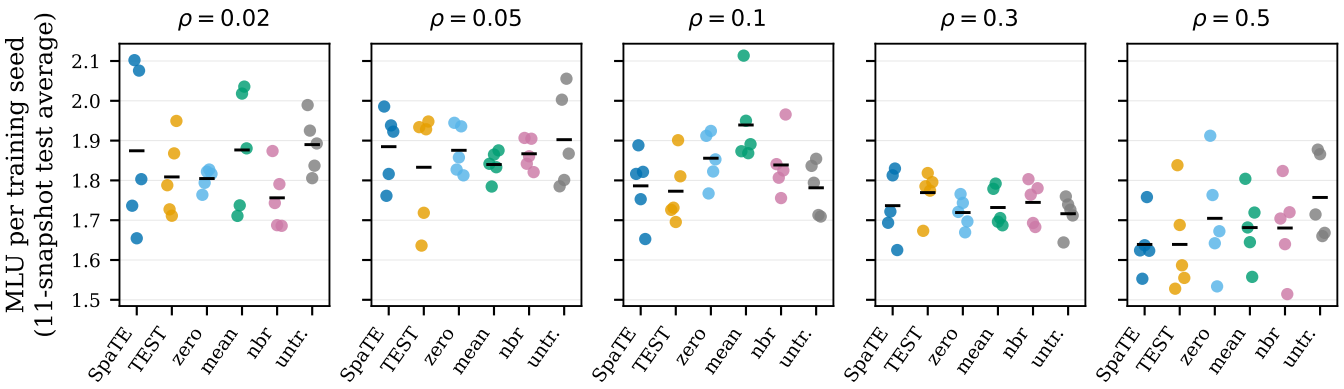

Fig. 7. Complete five-seed external comparison under the same deployed neighbor-assisted refinement. Each marker is one seed's average MLU over the 11 test snapshots; black bars mark means. The dispersion explains why column-wise rankings based only on a median are not reliable evidence of a general advantage.

[21] Y. Fan, J. Liu, P. Xie, and L. Liu, "Aether: Toward generalized traffic engineering with elastic multi-agent graph transformers," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.

[22] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "Path-based graph neural network for robust and resilient routing in distributed traffic engineering," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 2, pp. 422–436, 2025.

[23] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 661–678.

[24] Y. Lu, F. Sun, and Y. Zhao, "Virtual topology for LEO satellite networks based on earth-fixed footprint mode," *IEEE Communications Letters*, vol. 17, no. 2, pp. 357–360, 2013.

[25] M. Hu, M. Xiao, W. Xu, T. Deng, Y. Dong, and K. Peng, "Traffic engineering for software-defined LEO constellations," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5090–5103, 2022.

[26] L. Wei, T. T. Le, Y. Ji, M. Wang, Y. Liu, Y. Wang, and J. C. S. Lui, "Towards efficient traffic engineering via distributed optimization in large-scale LEO constellation," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 353–358.

[27] X. Liu, Z. Zhang, X. Qin, H. Zhou, and L. Zhao, "FlexSATE: Flexible and distributed traffic engineering with supervised learning in ultra-dense low-earth-orbit satellite networks," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 4466–4471.

[28] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, C. Ying, T. Liu, S. Yue, and H. Jin, "Scalable traffic allocation in dynamic networks via end-to-end imitation learning," *IEEE/ACM Transactions on Networking*, vol. 34, 2026.

[29] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, T. Liu, and H. Jin, "Learning to accelerate traffic allocation over large-scale networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.

[30] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American Statistical Association*, vol. 91, no. 433, 1996.

[31] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proceedings of ACM SIGMETRICS 2003*, 2003.

[32] Y. Zhang, M. Roughan, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices," in *Proceedings of the ACM SIGCOMM 2009 Conference*, 2009.

[33] C. Huo, D. Jin, Y. Li, D. He, Y.-B. Yang, and L. Wu, "T2-GNN: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[34] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[36] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[37] G. Giuliari, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 317–331.